

Lecture: 1011

Registers & Instruction Formats

MIPS32 & RISC-V: R, I, and J

Inside a 32-bit instruction: the registers it names, the fields it packs, and how to read the raw bits.

CSA222: Modern Computer Architecture

From Philosophy to Practice

L10 gave us RISC vs CISC in the abstract. Now we get concrete with a real RISC ISA.



RISC principles

Fixed-length, load/store, simple ops — the ideas from L10.



A real example

MIPS32: the cleanest teaching ISA, used in textbooks worldwide.



The register set

32 general-purpose registers, each with a conventional job.



The bit layouts

Three instruction formats — R, I, J — and how to decode them.

By the end you'll read a raw 32-bit hex instruction and write out the assembly it represents — by hand.

Today's Roadmap

Registers, then formats, then reading real machine code.

1

The Registers

MIPS's 32 registers and the conventions that give each one a job.



2

The Three Formats

R, I, and J — how every 32-bit instruction is laid out in fields.



3

Read & Write Bits

Decode a hex word to assembly, and hand-assemble the other way.

The goal: demystify machine code. A 32-bit instruction isn't magic — it's a handful of labelled bit-fields you can read.

Why MIPS for Teaching?

It's the ISA textbooks reach for — clean, regular, and free of real-world clutter.



Clean & fixed-width

Every instruction is exactly 32 bits, in one of just three formats. No exceptions to memorise.



Well-documented

Decades of textbooks (Patterson & Hennessy) use it. Endless references and worked examples.



MARS simulator

Write, run, and single-step MIPS assembly with instant feedback — no toolchain pain.



Ideas transfer

Learn MIPS and RISC-V, ARM, and others feel familiar — the concepts are the same.

We'll write MIPS in the MARS simulator this module — then map it onto RISC-V, which shares the same DNA.

1. The Register Set

32 general-purpose registers, each 32 bits wide — and a set of conventions that give every one a job.

32 Registers, 32 Bits Each

MIPS gives the programmer a bank of 32 fast registers, numbered \$0–\$31.

\$0	\$1	\$2	\$3	\$4	\$5	\$6	\$7
\$8	\$9	\$10	\$11	\$12	\$13	\$14	\$15
\$16	\$17	\$18	\$19	\$20	\$21	\$22	\$23
\$24	\$25	\$26	\$27	\$28	\$29	\$30	\$31

Each cell is one 32-bit register. Colours preview their conventional roles — the next slides give them names.

Registers are the fastest storage a program has — the register file from L7, now with a naming convention.

Register Conventions

The hardware treats them equally — but software agrees on a role for each. That's the calling convention.

\$zero

\$0

always 0

\$at

\$1

assembler temp

\$v0-\$v1

\$2-\$3

return values

\$a0-\$a3

\$4-\$7

function arguments

\$t0-\$t7

\$8-\$15

temporaries (caller-saved)

\$s0-\$s7

\$16-\$23

saved (callee-saved)

\$t8-\$t9

\$24-\$25

more temporaries

\$k0-\$k1

\$26-\$27

kernel / OS use

**\$gp / \$sp /
\$fp**

\$28-\$30

global / stack / frame ptr

\$ra

\$31

return address

The Special Registers

A few registers carry meanings you must never forget — starting with the most useful one of all.



\$zero (\$0)

Hardwired to 0. Reads always give 0; writes are ignored. Used constantly — e.g. move = add with \$zero.



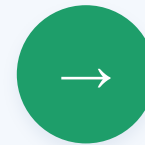
\$ra (\$31)

The return address. jal stores 'where to come back to' here so a function can return.



\$sp (\$29)

The stack pointer — tracks the top of the call stack for local variables and saved registers.



PC (separate)

The Program Counter isn't one of the 32 — it's a dedicated register holding the next instruction's address.

\$zero is the quiet hero: no separate 'load constant 0' or 'move' needed — just use \$zero as a source.

\$t vs \$s: The Saving Rules

The single most-tested convention: who is responsible for preserving a register across a function call.

\$t0-\$t9 • TEMPORARIES

caller-saved

- NOT preserved across a call.
- A called function may freely clobber them.
- If the caller needs the value, IT must save it first.
- Use for short-lived scratch values.

\$s0-\$s7 • SAVED

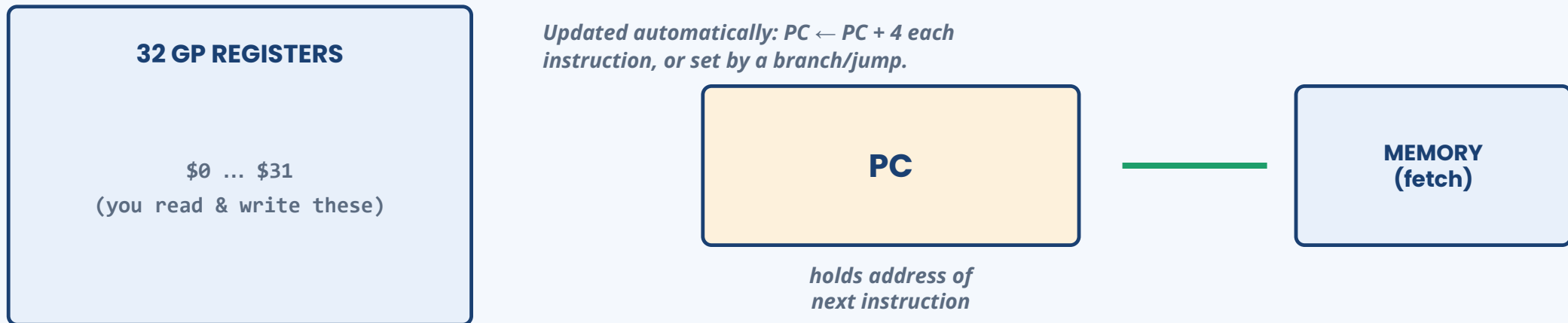
callee-saved

- Preserved across a call.
- A called function **MUST** restore them before returning.
- The caller can trust they survive.
- Use for values needed after a call.

Mnemonic: t = 'temporary, don't trust it'; s = 'saved, it survives'. This shows up in every exam.

The Program Counter Is Separate

The PC isn't one of the 32 GP registers — it's special-purpose, and you don't write it directly.



You change the PC indirectly — through branches (beq), jumps (j), and calls (jal) — never by writing to it like \$t0.

MIPS is 32-bit → each instruction is 4 bytes → PC advances by 4, not 1. Remember that '4'.

2. The Three Formats

Every MIPS instruction is 32 bits, arranged as one of just three field layouts: R, I, and J.

One Width, Three Layouts

Fixed 32-bit instructions are RISC's gift — but those 32 bits get divided up three different ways.

ALL INSTRUCTIONS ARE 32 BITS WIDE

31 ————— the 32 bits ————— 0
32 bits

The opcode (first 6 bits) tells the CPU which format to expect — then it knows how to slice the rest.



register ↔ register ALU ops



immediates, loads, stores, branches



jumps

The Three Formats Side by Side

Same 32 bits, three ways to carve them up. Notice all three share a 6-bit opcode.

R-FORMAT



I-FORMAT



J-FORMAT



The first 6 bits are always the opcode — the CPU reads them first to know which layout follows.

R-Format: Register-to-Register

For ALU ops on registers: add, sub, and, or, slt. Six fields, no memory address.



HOW TO READ IT

opcode = 0 → it's an R-type.
funct → the actual operation (add=0x20, sub=0x22).
rs, rt → the two source registers.
rd → the destination register.
shamt → shift amount (0 unless it's a shift).

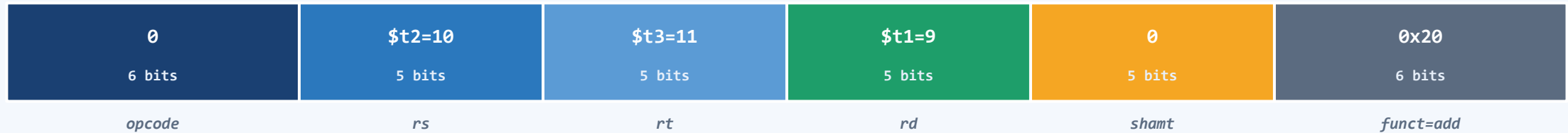
EXAMPLE

add \$t0, \$t1, \$t2

means: $\$t0 = \$t1 + \$t2$
rd=\$t0, rs=\$t1, rt=\$t2, funct=add.

Worked Decode: add \$t1, \$t2, \$t3

Watch the fields fill in — this is exactly the skill Activity 2 tests.



FIELD BY FIELD

`opcode = 000000` → R-type. `funct = 100000 (0x20)` → add.

`rs = 01010 = 10 = $t2` `rt = 01011 = 11 = $t3` `rd = 01001 = 9 = $t1`

Put together: `add $t1, $t2, $t3` → `$t1 = $t2 + $t3.`

I-Format: Immediates & Memory

For a constant, a load/store offset, or a branch distance — the last 16 bits hold a value, not a register.



THE 16-BIT IMMEDIATE

Only 3 fields — the immediate replaces rd, shamt & funct.

It's a 16-bit constant: an actual number, a memory offset, or a branch distance.

rt is usually the destination (loads, addi) or a compared source (branches).

EXAMPLES

```
addi $t0,$t1,5    $t0 = $t1 + 5
lw    $t0,8($sp)  load from sp+8
sw    $t0,8($sp)  store to sp+8
beq   $t0,$t1,L   branch if equal
```

J-Format: Jumps

The simplest layout — just an opcode and a big 26-bit address for `j` and `jal`.



`j` or `jal`

26-bit target (word address)

BUILDING THE TARGET

26 bits can't address all of 32-bit memory, so:

$\text{target} = (\text{PC}[31:28]) \mid (\text{address} \ll 2)$

$\ll 2$ gives byte address; top 4 bits come from the current PC.
Jumps stay within a 256 MB region.

THE TWO JUMPS

`j L` jump to label `L`

`jal L` jump AND save return
 address in `$ra` –
 this is a function call.

3. Reading Machine Code

The payoff: take a raw hex instruction and decode it to assembly — and assemble the other way, by hand.

Decode a Word (Activity 2)

Given `0x02324820` — split it into fields and recover the assembly.

STEP 1 – HEX TO BINARY

`0x02324820` = `0000 0010 0011 0010 0100 1000 0010 0000`

`opcode = 000000` → R-type, so slice: 6 / 5 / 5 / 5 / 5 / 6

STEP 2 – SPLIT THE FIELDS

000000 6 bits	10001 5 bits	10010 5 bits	01001 5 bits	00000 5 bits	100000 6 bits
<code>op=0</code>	<code>rs=17=\$s1</code>	<code>rt=18=\$s2</code>	<code>rd=9=\$t1</code>	<code>shamt=0</code>	<code>funct=0x20</code>

STEP 3 – ASSEMBLE: `funct 0x20 = add, rd=$t1, rs=$s1, rt=$s2` → **`add $t1, $s1, $s2`**

Decoding an I-Type

The same skill for a load — where the last 16 bits are one number, not two register fields.

DECODE: `lw $t1, 16($sp)` — opcode for `lw` is `0x23 (100011)`

100011

6 bits

Lw

11101

5 bits

rs=29=\$sp

01001

5 bits

rt=9=\$t1

00000000000010000

16 bits

imm=16

READING IT

opcode `0x23` → `lw` (load word).

`rs = $sp` → the base address.

`rt = $t1` → the destination.

`imm = 16` → the offset.

THE MEANING

`lw $t1, 16($sp)`

"load into `$t1` the word at
memory address `$sp + 16`."

The offset(base) pattern — `rs` is base, `imm` is offset.

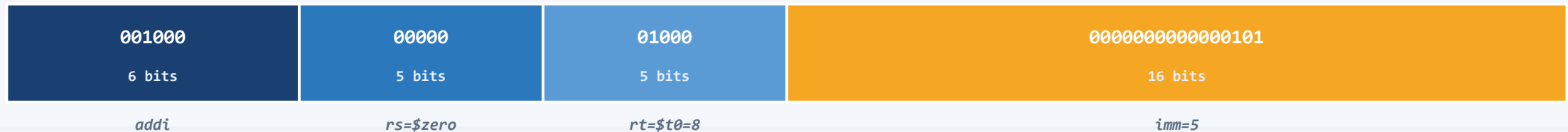
Hand-Assemble (Activity 3)

Now the reverse: turn `addi $t0, $zero, 5` into 32 bits of binary.

STEP 1 – IDENTIFY FIELDS

`addi` is I-format. `opcode(addi) = 0x08 = 001000`. `rs = $zero = 0`. `rt = $t0 = 8`. `imm = 5`.

STEP 2 – PLACE THE BITS



STEP 3 – REGROUP TO HEX: 0010 0000 0000 1000 0000 0000 0000 0101 = **0x20080005**

4. RISC-V's Modular ISA

Same format ideas, a different strategy: a tiny base plus optional extensions you bolt on as needed.

RISC-V: Start With a Tiny Base

RV32I is a minimal 32-bit integer ISA — small enough to implement in a classroom, complete enough to run real code.

RV32I — THE BASE

~40 instructions. 32 registers (x0–x31, x0 = zero, like \$zero).

Fixed 32-bit instructions, load/store architecture — the same RISC DNA as MIPS.

Formats: R, I, S, B, U, J — a refinement of MIPS's R/I/J with cleaner branch & immediate encodings.

MIPS vs RISC-V BASE

Both: 32 registers, fixed 32-bit, load/store.

MIPS: one fixed ISA — you get everything at once.

RISC-V: a minimal base you EXTEND. Want multiply? Add the M extension. Floats? Add F. Nothing you don't need.

That modularity is RISC-V's big idea — next slide.

The Extension Model

Build exactly the ISA your chip needs — base plus a menu of standard extensions.



e.g. RV32IMAFD = base + multiply + atomic + float + double. Pick your letters, get your ISA.

MIPS vs RISC-V: Same Idea, Refined

RISC-V learned from MIPS — keeping the good, fixing the awkward.

	MIPS32	RISC-V (RV32)
Registers	32 (\$0–\$31)	32 (x0–x31)
Zero register	\$zero (\$0)	x0
Instr. width	Fixed 32-bit	32-bit (+16-bit with C)
Formats	R, I, J	R, I, S, B, U, J
ISA model	One fixed set	Modular base + extensions
Licensing	Proprietary (MIPS Inc.)	Open & royalty-free

Learn one and the other is easy — the register-and-format model is the transferable skill, not the syntax.

Common Mistakes to Avoid

The slips that cost marks — and reveal shaky understanding.



Writing to \$zero

\$zero is hardwired to 0 — writes are silently ignored. Don't use it as a destination.



Confusing rd, rs, rt

In R-format, rd is the DESTINATION; rs & rt are sources. Order in binary ≠ order in assembly.



Forgetting PC + 4

Instructions are 4 bytes. The PC advances by 4, and branch targets are PC+4-relative.



\$t vs \$s saving

\$t registers are caller-saved (not preserved); \$s are callee-saved (preserved). Don't swap them.



Skipping sign extension

The 16-bit immediate is sign-extended to 32 bits. 0xFFFFB is -5, not 65531.



Forgetting the << 2

Branch and jump offsets are in WORDS, shifted left 2 to get bytes. Miss it and targets are wrong.

Key Takeaways

Five sentences — from register names to raw bits.

- 01 MIPS has 32 general-purpose registers, each with a conventional role: \$zero, \$a0–\$a3, \$t0–\$t9, \$s0–\$s7, \$sp, \$ra.
- 02 \$t registers are caller-saved (not preserved); \$s registers are callee-saved (preserved across calls).
- 03 Every MIPS instruction is 32 bits in one of three formats: R (register ops), I (immediate/memory/branch), J (jumps).
- 04 R = opcode | rs | rt | rd | shamt | funct; I = opcode | rs | rt | imm(16); J = opcode | address(26) — all sharing a 6-bit opcode.
- 05 RISC-V keeps the same model but is modular: a small RV32I base plus optional extensions (M, A, F, D, C, V).

Practice & Homework

Decode, assemble, and know your conventions.

01

Decode

Decode 0x012A4020 to MIPS assembly.
Show every field and the final mnemonic.

02

Hand-Assemble

Convert `lw $t1, 16($sp)` into 32-bit binary, then to hex. Show the fields.

03

Conventions

Explain the difference between \$t-registers and \$s-registers in the calling convention.

EXIT TICKET

1. Which fields does the R-format have, in order?
2. What value does \$zero always hold, and what happens if you write to it?
3. Name the six RISC-V extension letters and what each adds.

Rapid-Fire Recap

Three questions — shout the answers.

Q1**\$zero always holds...****0****Q2****R, I, J — how many bits is each instruction?****32****Q3****RISC-V's 'M' extension adds...****multiply**

All three instant? You can read machine code now — next we turn assembly into a running program.

END OF L11

You can read machine code now.

NEXT — L12

Assembly Programming & the MARS Simulator — from mnemonics to a running program: loops, branches, memory, and system calls.